Allocation and Coordination under Scarce Resources: Admissible Routing, Prices, Review Sets, and Containment for Gated Work

Ivo Matijašević

October 2026

Abstract

Earlier papers in this series decide which executor, gate stack, or mechanism is admissible for one step of work. They do not decide how scarce capacity is shared when many classes of work compete for it: a principal's attention, verification memory, external review lineages, rate-limited seats, and calendar time. This paper studies that allocation problem. We allocate only among options whose safety premises remain valid under the allocation actually chosen. We keep three distinctions: fluid feasibility versus executable schedules, marginal prices versus finite policy changes, and measured review value versus worst-case review value. In a fluid linear model, the feasible and optimal allocations do not depend on the executor's error law when bounds decide admissibility and the resource use, costs, demands, and capacities are law-independent or replaced by common uniform bounds. Feasibility has an exact certificate by nonnegative resource prices; subset cuts are exact for single-resource pools but insufficient for general resource bundles. Shadow prices value the principal's minutes, but a new option strictly lowers the optimum exactly when no old optimal dual prices it out, so a negative reduced cost at one dual is not enough. Choosing optional principal reviews is a knapsack. Review value under a fixed fault law is submodular; worst-case review value need not be, and a reviewer can have zero guaranteed value alone and positive guaranteed value in a coalition. Verifying an implementation does not identify the intended specification. Interface contracts bound severity: parallel escape routes add, and serial stages multiply only through conditional ceilings. With optional participation, achievable throughput does not fall as workers are added. Substituting a mechanization for scarce work needs feasibility in every resource and every period. Queueing models, a scheduling formulation of verification compute, and MaxWeight stability complete the picture, each under its stated assumptions. Many results are classical; we state the conditions under which they apply to gated work. These are mathematical models, not measurements.

1 Introduction

Part I of this series shows that a gate family with a common blind set leaves every executor with a verification floor [1]. Part II shows that gates generated within one family inherit its blind set [2]. Part III treats escape, cost, and correlated escape [3]. Part IV studies the move of work from executor calls to deterministic mechanisms [4]. Part V removes the constant-fault-rate assumption [5]. Part VI adds severity [6]. It defines when an option is admissible for a step: its worst-case expected loss, over the models compatible with the evidence, fits a severity-derived

© 2026 Ivo Matijašević. This work is licensed under a Creative Commons Attribution 4.0 International (CC BY 4.0) license, https://creativecommons.org/licenses/by/4.0/. Preprint, DOI 10.5281/zenodo.23449199.

budget. That decision uses only floors, ceilings, and certified levels, so it holds for every error law within those bounds.

These papers decide one step at a time. A working system runs many classes of work at once, and they compete for capacity. Some resources are scarce: the principal's attention, the memory of a verification machine, the capacity of an external review lineage, rate-limited seats, and calendar time. This paper asks which worker or mechanism should receive which unit when such resources bind, and how the organization of the work (decomposition, interfaces, parallelism) changes risk and capacity.

The answer rests on one rule and three distinctions. The rule: allocate only among options whose admissibility holds under the allocation actually chosen, including its concurrency and review arrangement. The distinctions: fluid feasibility versus executable schedules; marginal prices versus finite policy changes; and measured review value versus worst-case review value.

Contributions.

1. A fluid allocation model over admissible options, with the qualifications that admissibility needs (Definition 1), and its invariance to the executor's error law (Theorem 2).
2. Capacity and feasibility: the bottleneck bound, Hall's condition for single-resource pools, and an exact price certificate for resource bundles (Proposition 4, Theorems 6 and 8).
3. Shadow prices with the exact condition under which a new option helps, and a counterexample to the unrestricted reduced-cost test (Theorem 10, Example 11); few split classes at a fluid optimum (Proposition 12).
4. The principal's attention: a knapsack with the conditions it needs (Proposition 13); submodular fixed-law review value and non-submodular robust review value (Theorem 15); the principal as producer (Corollary 16); and the gap between verification and intent (Proposition 17).
5. Containment: modularity and serial containment bounds on severity (Theorems 18 and 20); parallelism on coupled work (Proposition 21); review lineages as capacity (Corollary 22).
6. Substitution and scheduling: per-resource, per-period substitution feasibility (Proposition 23); queueing at a scarce gate (Proposition 25); a scheduling formulation of verification compute (Proposition 27); and MaxWeight stability over admissible configurations (Theorem 30).

Scope and status of the results. A result whose heading carries a citation is a known result. We state it in this paper's notation, with the conditions that our use needs. Every other result is proven here in the stated model; that label does not claim originality. No result in this paper is a conjecture. The contribution is the set of conditions under which the classical tools apply to gated work, and the counterexamples that mark where they fail.

2 Model

Background from the series. A unit of work is produced by an executor and passes a stack of gates. An *escape* is a faulty unit that passes every gate. An executor's conditional escape probability, given the full history, has a *floor* and a *ceiling*. A process whose conditional escape lies between the bounds ℓ and u at every history belongs to the class $\mathcal{P}(\ell, u)$; mixtures of executors and adaptive routing among them stay in the class [6]. A gate accepts every fault in its blind set. A blind set is *complete* if it is exactly the set of accepted faults. A conjunctive review stack accepts

a fault only if every member accepts it, so with complete blind sets a producer with badness p_e and fault law μ_e escapes the reviewers $r_1, \dots, r_k$ with probability $p_e \mu_e(\bigcap_j \Sigma_{r_j})$ [2, 6]. A step has a severity, and an option is *admissible* for the step when its severity-weighted worst-case expected loss, over the models compatible with the evidence, fits a tolerated loss. Admissibility has a robust reading, for a deployed build, and an ex-ante reading, for a build-and-evidence experiment with a statistical certificate [6]. On a directed acyclic dependency graph, the *structural severity* of an escape at node v is $S(v) = \sum_w \rho(v \to w)\, c(w)\, \iota(w)$, where $\rho(v \to w)$ is the probability that the fault reaches w, $c(w)$ the cost of reaching w, and $\iota(w)$ the probability that this damage is not undone [6].

The allocation model.

Definition 1 (Allocation model)**.** Unit classes k have demand $\lambda_k \geq 0$ per period. $\mathcal{A}_k$ is the finite set of options admissible for k under an explicitly chosen reading of admissibility [6]. Option a for class k consumes $D_{r,ak} \geq 0$ of resource r per unit and costs c_{ak}. Resource r has capacity κ_r per period. The *fluid model* is

$$x_{ak} \geq 0, \qquad \sum_{a \in \mathcal{A}_k} x_{ak} = \lambda_k, \qquad \sum_{k,a} D_{r,ak} x_{ak} \leq \kappa_r. \tag{F}$$

Four qualifications belong to the definition.

(1) Admissibility must hold on the routed tasks and histories, including the chosen concurrency and review arrangement. Benchmark averages do not establish it.

(2) An ex-ante statistical certificate keeps that label; it does not become robust admissibility.

(3) Attention time and CPU time enter as workload rates. Memory, simultaneous seats, precedence, and deadlines need scheduling constraints beyond (F) (Proposition 27).

(4) Executor error laws may stay unknown within the bounds of [6]. Arrival and service laws need separate assumptions wherever a queueing result uses them.

We write D_{ak} for the vector $(D_{r,ak})_r$ and κ for $(\kappa_r)_r$.

3 The allocation backbone

Theorem 2 (Admissible allocation is invariant to the executor's error law)**.** *Suppose that bounds of the kind in [6] decide every option's admissibility. Suppose also that the coefficients D, c, λ, κ do not depend on the error law, or are replaced by bounds that hold for every law in the class. Then the feasible set and the optimal allocations of* (F) *are the same for every executor error law, dependence structure, and routing mixture that satisfies those bounds.*

Proof. The program's data are λ, κ, D, c, and the sets $\mathcal{A}_k$. The bounds fix each $\mathcal{A}_k$, by the admissibility theorem and the mixture lemma of [6]. Equal data give equal feasible and optimal sets. □

Remark 3. Retries consume attempts, so D can depend on the error law. Then the coefficients need interval data and the worst case. Admissibility must hold under the chosen concurrency (Definition 1(1)). Replacing law-dependent coefficients by bounds that hold for every law is the worst-case reformulation of inexact linear programs [75]; robust linear programming generalizes it [15].

Proposition 4 (Bottleneck bound; operational laws [20, 25])**.** *If completed class-k units require long-run average demand $D_{r,k} > 0$ of resource r, then the class-k throughput satisfies $X_k \le \min_{r:D_{r,k}>0} \kappa_r / D_{r,k}$. Other classes' use of r tightens the bound. If a fraction $f > 0$ of units needs principal review with mean time s given review, then $Xfs \le \kappa_{\mathrm{pr}}$.*

Proof. Over a horizon T, required work cannot exceed capacity, apart from boundary work of order $o(T)$. Divide by T. □

Remark 5. With $\kappa_{\mathrm{pr}} = 120$ principal minutes per period, $f = 0.1$, and $s = 5$ minutes, the ceiling is $X \le 240$ units per period; halving f at the same s doubles it to 480. Admissible mechanization does not imply that f falls. It can keep every principal approval, add validation, or raise exceptions. A lower f with a longer s can raise fs.

Theorem 6 (Feasibility of admissible routing for single-resource pools; Hall [39], Gale [33], Ford and Fulkerson [30])**.** *Suppose each option is a pool with capacity b_a, each unit uses one unit of its chosen pool, and no other shared constraint exists. Write $\mathcal{A}(T) = \bigcup_{k\in T} \mathcal{A}_k$. A feasible routing exists if and only if*

$$\sum_{k\in T} \lambda_k \le \sum_{a\in\mathcal{A}(T)} b_a \qquad \text{for every set } T \text{ of classes.}$$

Integral data admit an integral routing. A minimum cut in the source–class–pool–sink network names a violating set.

Proof. Use max-flow min-cut. A finite cut is fixed by the set T of classes on the source side, and its capacity is $\sum_{k\notin T} \lambda_k + \sum_{a\in\mathcal{A}(T)} b_a$. The routing exists exactly when the maximum flow equals $\sum_k \lambda_k$. Integrality of the maximum flow gives the integral routing. □

The condition is the bipartite case of Gale's supply–demand feasibility theorem [33], which extends Hall's theorem.

Example 7 (Shared gate)**.** The theorem is false when options share a resource. Take one class with demand 2 and two options, each a pool of capacity 1, that both consume one principal gate of capacity 1. The pool condition holds ($2 \le 1 + 1$), but the gate can carry only one unit. Theorem 8 covers this case.

Theorem 8 (Exact feasibility certificate for resource bundles; Farkas [28], LP duality [16])**.** *Let every $\mathcal{A}_k$ be finite and nonempty. Then (F) is feasible if and only if, for every price vector $p \ge 0$,*

$$\sum_k \lambda_k \min_{a\in\mathcal{A}_k} p^\top D_{ak} \;\le\; p^\top \kappa.$$

Infeasibility therefore has a certificate of nonnegative resource weights, not necessarily a subset cut.

Proof. Necessity: weight the capacity rows by p and use $\sum_a x_{ak} = \lambda_k$. Sufficiency: the dual of the feasibility LP is

$$\max\{y^\top \lambda - p^\top \kappa :\; y_k \le p^\top D_{ak}\ \forall a \in \mathcal{A}_k,\ p \ge 0\}.$$

The primal is infeasible if and only if the dual is unbounded, if and only if some (y, p) has positive value, if and only if the displayed inequality fails for some $p \ge 0$. □

Example 9 (Coordinatewise tests are not enough)**.** One unit has options with demand vectors $(2, 0)$ and $(0, 2)$, and the capacities are $(0.6, 0.6)$. For each resource separately, some option uses none of it, so both coordinatewise tests pass. The price $p = (1, 1)$ gives $\min_a p^\top D_a = 2 > 1.2 = p^\top \kappa$ and certifies infeasibility. A certified additive damage allowance can enter the model as one more resource row.

Suppose the resource rows are the edges of a capacitated network, each demand vector D_{ak} is the edge-incidence vector of a path, and each class's options are all paths between its source and its sink. Then $\min_a p^\top D_{ak}$ is a shortest-path length under the edge lengths p, and Theorem 8 becomes the multicommodity feasibility theorem of Onaga and Kakusho and of Iri [70, 43]. With only some paths admissible, the minimum is over the admissible paths.

Theorem 10 (Shadow prices of scarce resources; LP duality and sensitivity [18])**.** *Let* $V(\lambda, \kappa) = \min\{c^\top x : \ (\mathrm{F})\}$*, with dual*

$$\max\{y^\top \lambda - p^\top \kappa : \ y_k \le c_{ak} + p^\top D_{ak}, \ p \ge 0\}.$$

(i) At an optimal dual (y, p)*,* $V(\lambda, \kappa+\Delta) \ge V(\lambda, \kappa) - p^\top \Delta$*. Where* V *is differentiable,* $\partial V/\partial \kappa_r = -p_r$*. At a kink, keep the set of optimal duals.*

(ii) If the old problem is feasible, adding options (columns) strictly lowers the optimum if and only if no *optimal dual of the old problem satisfies all the new columns' dual constraints. Without feasibility the condition can hold vacuously. With a unique optimal dual, a negative reduced cost* $c + p^\top D - y_k < 0$ *is sufficient.*

(iii) Changes to demand, coefficients, admissible sets, or available options are discrete and in general need reoptimization. The prices give only local or bounding information about them.

Proof. Part (i) is weak and strong duality. Part (ii): the new optimum equals the old one if and only if some old optimal dual remains feasible for the enlarged dual. Part (iii) states the scope of (i) and (ii): both are local or bounding statements. □

Example 11 (The unrestricted reduced-cost test fails)**.** Take the old problem $x = 1$, $x \le 1$, with cost 0. Every $y = p \ge 0$ is dual-optimal. A new option with zero cost and zero resource use has reduced cost $0 + 0 - 1 = -1$ at $y = p = 1$, yet the optimum stays 0.

The old problem is primal degenerate, so its optimal dual is not unique. That the prices of a single dual solution can then mislead is known [11, 8, 32, 44], and directional marginal values are extrema over the set of optimal duals [62, 77]. Part (ii) states this fact for added columns. The price p_{pr} values a principal minute at the current load and admissibility. A candidate mechanization can then be compared across all scarce resources in one currency, with the limits of parts (ii) and (iii).

Proposition 12 (Few coupling constraints imply few split classes; basic feasible solutions [16], Lenstra, Shmoys, and Tardos [57])**.** *A feasible bounded instance of* (F) *with* K *classes of positive demand and* m *resource rows has an optimal solution with at most* $K + m$ *positive coordinates. Hence at most* m *classes are split across options. At a vertex with* b *binding independent resource rows, at most* b *classes are split.*

Proof. A linear objective attains its optimum at a vertex. A vertex has at most as many positive coordinates as independent active equality rows. Each class with positive demand needs at least one positive coordinate, so at most m (or b) classes have two or more. □

A fluid optimum therefore assigns most classes to one option. Randomized routing is needed for at most as many classes as there are binding scarce resources. The same vertex count bounds the fractional jobs in the linear relaxation of scheduling on unrelated parallel machines [57], and it gives at most three split categories in a recent model of AI review before human approval [54].

4 The principal's attention and review sets

Proposition 13 (Principal-attention allocation is a knapsack; Dantzig [24], Kellerer, Pferschy, and Pisinger [46])**.** *Suppose optional reviews have fixed avoided losses $v_i \geq 0$ and times $t_i > 0$, effects are additive across units, and there are no setup or interaction effects. Then choosing reviews within principal time H is a knapsack problem.*

(a) The ratio order v_i/t_i solves the fractional problem.

(b) For the integer problem, first discard reviews with $t_i > H$. Then the better of the ratio prefix of the remaining reviews and the best remaining single review is a 1/2-approximation (the standard modified greedy).

(c) Without the discard step the guarantee fails. With $H = 3$, one review $(t, v) = (4, 100)$, and three reviews $(1, 1)$, an undiscarded ratio prefix is empty, and the best feasible single review gives 1 against an optimum of 3.

(d) With complete blind sets, constant severity, and review that only suppresses faults, $v_i = S_i p_i \mu_i(\Sigma_i \setminus \Sigma_{\mathrm{pr}})$. Here $p_i\mu_i$ is the unit's escape law through its existing stack, Σ_i is the blind set of that stack, Σ_{pr} is the principal's blind set, and S_i is the unit's severity.

Proof. Parts (a) and (b) are the classical results [24, 46]. Part (c): the review $(4, 100)$ has the largest ratio and does not fit, so the prefix stops before it. Part (c) shows why the discard step, often stated as the assumption that every item fits, is needed. Part (d): the principal's review removes the escaped faults outside Σ_{pr}, each at damage S_i. □

Remark 14 (Conditions). Reviews required for admissibility are not optional and are scheduled first. Fault-dependent damage needs integration, not a common severity. Lower bounds $\underline{v}_i$ give a conservative objective, but a 1/2-approximation to that objective is not a 1/2-approximation to the actual benefit. Overlapping coverage breaks additivity (Theorem 15).

Theorem 15 (Fixed-law review value is submodular; robust review value need not be; Nemhauser, Wolsey, and Fisher [66], Krause et al. [50])**.** *Fix a fault measure ν, a damage $h \geq 0$, and reviewer rejection sets C_r, where review only suppresses faults. Then*

$$F_\nu(A) = \int h\, \mathbf{1}\Big\{z \in \bigcup_{r \in A} C_r\Big\}\, d\nu(z)$$

is monotone submodular, so greedy selection of k reviewers of equal cost achieves a fraction $1 - 1/e$ of the optimum [66]. The robust value $F_{\mathrm{rob}}(A) = \inf_{\nu \in \mathcal{U}} F_\nu(A)$, over a set $\mathcal{U}$ of fault measures, need not be submodular.

Proof. Fixed law: the marginal value of r given a coalition is the damage-weighted mass that r catches and the coalition does not. This mass shrinks as the coalition grows. The $1 - 1/e$ guarantee is the theorem of Nemhauser, Wolsey, and Fisher [66]. Robust counterexample: take faults x, y, unit damage, $C_1 = \{x\}$, $C_2 = \{y\}$, and $\mathcal{U} = \{\delta_x, \delta_y\}$. Then $F_{\mathrm{rob}}(\varnothing) = F_{\mathrm{rob}}(\{1\}) = F_{\mathrm{rob}}(\{2\}) = 0$ and $F_{\mathrm{rob}}(\{1, 2\}) = 1$, so $F_{\mathrm{rob}}(\{1\}) + F_{\mathrm{rob}}(\{2\}) < F_{\mathrm{rob}}(\{1, 2\}) + F_{\mathrm{rob}}(\varnothing)$. □

The fixed-law value is a weighted coverage function, and the $1 - 1/e$ guarantee is tight for coverage [29]. The robust half is known: a pointwise minimum of monotone submodular functions is in general not submodular, and robust selection is hard to approximate [50]; see also [72]. Our counterexample has the same two-point form as theirs. A reviewer can thus have zero guaranteed value alone and positive guaranteed value in a coalition. Certified reviewer *sets* should be allocated; reviewers should not be ranked by singleton lower bounds.

Corollary 16 (The principal as producer inherits common blind sets)**.** *Suppose the principal produces bad outputs with probability p_{pr} and conditional fault law μ_{pr}, and every reviewer in an allowed coalition accepts a common set B. Then the first-attempt escape floor of the principal's work is at least $p_{\mathrm{pr}}\mu_{\mathrm{pr}}(B)$. Self-review by the principal, or more members that keep B, cannot lower it, provided the principal's own review also accepts B. Production by the principal alone does not establish that premise.*

Proof. Every fault in B passes every member. Apply the conjunctive-review result of [6] with the principal as producer. □

Proposition 17 (Implementation verification does not identify intent; Le Cam's two-point bound [52])**.** *Suppose two intended specifications induce the same available transcript but require opposite binary actions, with loss S for the wrong action. Then every transcript-based rule has worst-case expected loss at least $S/2$. This holds even if the rule proves that its implementation satisfies the recorded specification exactly.*

Proof. A rule sees the same transcript under both specifications, so it chooses action 1 with the same probability q. Its losses are Sq and $S(1-q)$, and $\max\{Sq, S(1-q)\} \geq S/2$. □

This is the exact-twin case of the twin regret of [6], with equal gaps, and so an instance of Le Cam's two-point bound [52]. The two specifications are observationally equivalent in the sense of Koopmans [49]. The split between building the right product and building the product right is Boehm's distinction between validation and verification [17]. Establishing the operative specification (authority) and showing that an action meets it (verification) are different objects. Before any theorem about the principal's errors, one must define whether the principal's declaration *is* the specification or reports a latent intent imperfectly.

5 Containment and organization

In this section, condition on an escape at node v. Let $S(v)$ be its expected damage, and $S_{\mathrm{in}}(v)$ the expected damage inside v's module.

Theorem 18 (Modularity bounds severity)**.** *(a)* General form. *Let C be a set of interfaces that separates v's module from the outside, and let E_e be the event that the fault's cascade ever crosses interface e. Suppose $\mathbb{P}(E_e \mid \text{escape at } v, \text{routed history}, \text{fault type}) \leq \varepsilon_e$ uniformly over compatible models. Suppose also that outside damage is 0 without a crossing and at most C_{out} pathwise otherwise. Then*

$$S(v) \leq S_{\mathrm{in}}(v) + C_{\mathrm{out}} \min\Big(1, \sum_{e \in C} \varepsilon_e\Big).$$

No independence is needed.

(b) Model form. *In the independent-edge model of [6] on a directed acyclic graph, let M be v's module. Suppose every path from v to a node outside M uses an edge of E_{cut}, and each such edge transmits with probability at most ε_e. Write $S_M(v) = \sum_{w \in M} \rho(v \to w) c(w) \iota(w)$ and $C_{\mathrm{out}} = \sum_{w \notin M} c(w)\iota(w)$. Then*

$$S(v) \leq S_M(v) + C_{\mathrm{out}} \sum_{e \in E_{\mathrm{cut}}} \varepsilon_e \, \rho(v \to \mathrm{tail}(e)).$$

Proof. (a) Outside damage is at most $C_{\text{out}}\mathbf{1}_{\bigcup_e E_e}$. Take expectations and use the union bound and $\mathbb{P} \le 1$. (b) Reaching $w \notin M$ requires reaching $\text{tail}(e)$ for some $e \in E_{\text{cut}}$ and e transmitting. In a directed acyclic graph with independent edges these two events are independent, because reaching $\text{tail}(e)$ does not use e. Apply the union bound, multiply by $c(w)\iota(w)$, and sum over $w \notin M$. □

Remark 19 (Conditions). A pass rate on a reference distribution is not a conditional crossing bound for a fault routed to the interface. A per-attempt bound is not an "ever crosses" bound under retries. Contracts must intercept the relevant semantic faults, be enforced, and have no bypass. Smaller severity weakly enlarges the budget of [6]; it does not guarantee a cheaper admissible option, or that the interface overhead pays.

Part (a) is the union-bound form of the cut bounds of reliability theory [27], without their independence assumption. The ceilings ε_e play the role of the error permeabilities of software modules [41, 42], and the module bounded by C is a fault-containment region [51].

Theorem 20 (Serial containment multiplies conditional leakage ceilings)**.** *Suppose every route from an escaped fault to outside damage must survive ordered stages $1, \dots, L$. Let B_0 be the escape and $B_j \subseteq B_{j-1}$ the event of surviving through stage j, and suppose $\mathbb{P}(B_j \mid B_{j-1}, \text{history entering } j) \le \varepsilon_j$ uniformly. Then $\mathbb{P}(B_L) \le \prod_j \varepsilon_j$, and, with C_{out} as in Theorem 18(a),*

$$S(v) \le S_{\text{in}}(v) + C_{\text{out}} \prod_{j=1}^{L} \varepsilon_j.$$

Proof. $\mathbb{P}(B_j) \le \varepsilon_j \mathbb{P}(B_{j-1})$; iterate. Outside damage is at most $C_{\text{out}}\mathbf{1}_{B_L}$; take expectations. □

Parallel escape routes add (Theorem 18, union bound). Serial stages multiply (Theorem 20, conditional bounds). Marginal ceilings do not multiply: two stages with marginal failure probability ε but identical failure events give joint failure ε, not ε^2. Renaming reviewers as stages buys no conditional guarantee. For independent layers, the product is the layer-multiplication rule of layer-of-protection analysis [56]; Theorem 20 replaces independence by uniform conditional ceilings. The failure of marginal multiplication is the coincident-failure effect of diverse software [26, 55].

Proposition 21 (Parallelism on coupled work; universal scalability law [37])**.** *(a) If participating workers are optional, so that extra workers may stay idle, the best achievable throughput is nondecreasing in the number of available workers.*

(b) Suppose throughput follows $X(n) = X(1)\, n/[1 + \alpha(n-1) + \beta n(n-1)]$ with $0 \le \alpha < 1$ and $\beta > 0$, and $X(1) > 0$. Its continuous maximum is at $n^* = \max\left(1, \sqrt{(1-\alpha)/\beta}\right)$. The law is not globally concave; its tail is convex.

(c) With $\beta = 0$ and $X(1) > 0$ there is no finite maximizer. For $0 < \alpha < 1$, throughput increases to the limit $X(1)/\alpha$; for $\alpha = 0$, it grows linearly without bound.

Proof. (a) Idling the extra workers reproduces any smaller configuration. (b) The sign of the derivative is the sign of $1 - \alpha - \beta n^2$. X is rational, so its Laurent expansion at infinity converges and may be differentiated term by term: $X(n) = X(1)/(\beta n) - X(1)(\alpha - \beta)/(\beta^2 n^2) + O(n^{-3})$, hence $X''(n) = 2X(1)/(\beta n^3) + O(n^{-4}) > 0$ for all sufficiently large n. (c) With $\beta = 0$, $X(n) = X(1)n/[1 - \alpha + \alpha n]$, which increases in n and tends to $X(1)/\alpha$ for $\alpha > 0$ and equals $X(1)n$ for $\alpha = 0$. □

Part (a) is proven here. The law in (b) and (c) is Gunther's model [37], and Gunther also gives its maximizer and its reduction to Amdahl's law at $\beta = 0$ [38]; Amdahl stated that ceiling in words [10]. For example, $\alpha = 0.1$ and $\beta = 0.01$ give $n^* = \sqrt{90} \approx 9.49$, and the integer worker counts 9 and 10 tie, both giving $X(n)/X(1) = 25/7$. A falling throughput curve describes a policy that makes extra workers participate. It is not a limit on what is achievable. Pairwise coordination cost does not by itself fix the law.

Corollary 22 (Review lineages as capacity)**.** *Treat each certified review bundle as an option with a resource-demand vector. Its throughput obeys Proposition 4 and Theorem 8, and Theorem 6 when the pools are interchangeable single resources. Distinct lineages do not by themselves give independence or complementary blind sets. What matters is the certified acceptance intersection for the producer and the routed workload [6].*

Proof. A bundle with a demand vector is an option of Definition 1; apply the cited results. The escape of a conjunctive stack depends on the intersection of its members' acceptance sets, not on their lineage labels [6]. □

6 Substitution, scheduling, and stability

Proposition 23 (Lifecycle substitution needs vector, time-indexed feasibility)**.** *Over a horizon, let s_r be the current slack of resource r, with the rest of the allocation fixed. Replace n units by a mechanization with fixed demand F_r (build, validation) and recurring demand d_r^{cl} in place of d_r^{op}. The replacement is feasible if and only if*

$$F_r + n(d_r^{\mathrm{cl}} - d_r^{\mathrm{op}}) \le s_r \qquad \text{for every } r,$$

and in every period when demands are time-indexed.

Proof. Substitute the changed demand into each capacity row. □

Multi-period capacity planning keeps a capacity row for each resource and each period [58].

Example 24 (Lifecycle totals mislead)**.** Let the principal's capacity be 10 in each of two periods, the old demand $(10, 10)$, and the new demand $(15, 0)$. Total demand falls from 20 to 15, but period one is infeasible. A slack resource can also become the new bottleneck. Reallocation elsewhere can compensate. Fixed costs and integer commitments lie outside the marginal test of Theorem 10.

Proposition 25 (Queueing at a scarce gate; Harchol-Balter [40], Kingman [47, 48], Cox and Smith [23])**.** *Each item holds under its stated model.*

(1) M/M/1, first come first served. *With Poisson arrivals at rate λ, exponential service at rate $\mu > \lambda$, and one server, $\mathbb{E}W = 1/(\mu - \lambda)$ and $\mathbb{E}W_q = \rho/(\mu - \lambda)$, with $\rho = \lambda/\mu$.*

(2) Kingman (GI/GI/1). $\mathbb{E}W_q \approx \frac{\rho}{1-\rho} \cdot \frac{c_a^2 + c_s^2}{2} \mathbb{E}S$, *where c_a and c_s are the coefficients of variation of interarrival and service times. This is a heavy-traffic approximation, not a universal bound.*

(3) The $c\mu$ rule. *With Poisson class arrivals, exponential class service, one preemptive-resume server, no switching cost, and constant linear holding costs c_k, serving the largest $c_k\mu_k$ first is optimal.*

(4) Batching. *With setup time h, batch size b, and per-unit service time s, saturated capacity is $b/(h + bs)$. Under Poisson arrivals, forming a batch adds mean per-unit delay $(b - 1)/(2\lambda)$.*

Proof. Items (1)–(3) are classical [40, 47, 48, 23, 19]. Item (4): a saturated cycle processes b units in time $h+bs$. In a batch, the i-th arrival waits for $b-i$ further arrivals, each after a mean interarrival time $1/\lambda$; the average of $(b-i)/\lambda$ over $i=1,\dots,b$ is $(b-1)/(2\lambda)$. Batch-service queues go back to Bailey and Neuts [12, 67]; item (4) gives only the elementary capacity and formation-delay terms. □

Remark 26 (Caveats). Severity is not automatically a holding-cost rate. Work awaiting safe release incurs delay cost, while work already exposed accumulates damage. The invariance of [6] to error laws gives no invariance to arrival or service laws.

Proposition 27 (Verification compute is resource-constrained scheduling; Pritsker, Watters, and Wolfe [73])**.** *Suppose durations, machine eligibility, memory needs, releases, and deadlines are known. Then allocating proof jobs is a cumulative-resource scheduling problem. Let $z_{jmt} \in \{0,1\}$ indicate that job j starts on machine m at slot t, let p_{jm} be its duration on m, m_j its memory need, and M_m the memory of machine m. Memory requires*

$$\sum_{j,t:\ t\le\tau<t+p_{jm}} m_j z_{jmt} \le M_m \qquad \text{for every } m \text{ and } \tau,$$

together with assignment, precedence, and deadline constraints. A cloud option adds placements and costs. The LP relaxation gives lower bounds and prices in the sense of Theorem 10.

Proof. A job started at slot t on machine m occupies m_j units of memory in the slots $t,\dots,t+p_{jm}-1$, so the displayed sum is the memory in use at slot τ. Relaxing integrality enlarges the feasible set, so the relaxation's optimal cost is a lower bound, and its dual solutions are prices. □

Example 28 (Memory-time totals are only a relaxation)**.** Three one-slot jobs each need 6 units of memory on a 10-unit machine. No two of them can overlap, since $2\cdot 6 = 12 > 10$. Yet a rate of $1/2$ for each job passes the aggregate test $3\cdot\frac{1}{2}\cdot 6 = 9 \le 10$, although a total rate of $3/2 > 1$ requires overlap in some slot.

The time-indexed formulation is that of Pritsker, Watters, and Wolfe [73]. Memory is a cumulative resource [7], and the aggregate test is an energy-type relaxation of the kind studied in cumulative scheduling [14].

Remark 29 (Caveats). Covered severity is not additive when proof scopes overlap. A solver run is an admissible verification option only under its soundness, artifact-binding, and enforcement premises. A rule that all solver runs go to the cloud is a policy constraint, not an optimality result.

Theorem 30 (MaxWeight scheduling of feasible configurations is throughput-optimal; Tassiulas and Ephremides [76])**.** *Consider slotted queues $Q_k(t+1) = [Q_k(t) - s_k(t)]_+ + A_k(t)$. Let $\mathcal{M}$ be a finite set of service vectors that are simultaneously feasible, resource compatibility included. Let the arrivals be i.i.d. with mean λ and finite second moments, independent of a finite initial backlog with $\mathbb{E}\|Q(0)\|_2^2 < \infty$. If some $\bar{s} \in \operatorname{conv}\mathcal{M}$ has $\bar{s} \ge \lambda + \epsilon\mathbf{1}$ with $\epsilon > 0$, then MaxWeight, $s(t) \in \arg\max_{s\in\mathcal{M}} Q(t)^\top s$, is strongly stable, with*

$$\limsup_{T\to\infty} \frac{1}{T}\sum_{t<T} \mathbb{E}\|Q(t)\|_1 \le \frac{B}{2\epsilon}, \qquad B = \mathbb{E}\|A\|_2^2 + \max_{s\in\mathcal{M}} \|s\|_2^2.$$

Proof sketch. Use quadratic Lyapunov drift. One slot gives $\mathbb{E}[\|Q(t+1)\|_2^2 - \|Q(t)\|_2^2 \mid Q(t)] \le B + 2Q(t)^\top(\lambda - s(t))$. MaxWeight maximizes $Q^\top s$ over $\mathcal{M}$, hence over $\operatorname{conv}\mathcal{M}$, so $Q^\top s(t) \ge Q^\top \bar{s} \ge Q^\top\lambda + \epsilon\|Q\|_1$. The drift is therefore at most $B - 2\epsilon\|Q(t)\|_1$. Take expectations, telescope over $t<T$, use $\mathbb{E}\|Q(0)\|_2^2 < \infty$, and divide by $2\epsilon T$. The explicit bound is the standard Lyapunov-drift form [34, 64]. □

Build $\mathcal{M}$ from options whose safety bounds hold under each mode's concurrency. Rework that depends on errors belongs in the arrival and service model. Stability does not enforce the lifetime consequence budget of [6]. Throughput sustained indefinitely at positive risk and a finite lifetime budget can be incompatible.

7 Consequences for scheduler design

The results suggest the following guidance. It is design guidance, not law. Each item is only as strong as the result it cites, and holds only under that result's conditions.

1. *Filter first, then allocate.* Admit an option to $\mathcal{A}_k$ only if its admissibility holds under the allocation that will run it: the routed tasks, the concurrency, and the review arrangement (Definition 1). Then the allocation does not need the executors' error laws (Theorem 2). If retries make demand depend on the error law, plan against interval data and the worst case.

2. *Check feasibility jointly.* Test feasibility with a price certificate over all resources (Theorem 8). Resource-by-resource tests can pass an infeasible plan (Example 9). Subset cuts are exact for single-resource pools but insufficient for general resource bundles (Theorem 6, Example 7).

3. *Use prices locally; reoptimize for discrete changes.* Dual prices value scarce resources, including principal minutes, in one currency. To decide whether a new option helps, check all old optimal duals, or reoptimize (Theorem 10, Example 11).

4. *Expect few split classes.* A fluid optimum splits at most as many classes as there are binding resources (Proposition 12). Randomized routing can be confined to those classes.

5. *Schedule required principal reviews first.* Then fill the remaining principal time with optional reviews by the ratio rule, after discarding reviews that do not fit, and compare with the best single review (Proposition 13). A guarantee on lower-bound values is not a guarantee on actual benefit.

6. *Allocate certified reviewer sets.* Do not rank reviewers by singleton lower bounds; a reviewer with no guaranteed value alone can complete a coalition (Theorem 15). Lineage labels do not certify complementary blind sets (Corollary 22).

7. *Keep authority and verification apart.* A proof that work meets the recorded specification does not show that the specification is the intended one (Proposition 17). Work the principal produces keeps the common blind set of its reviewers (Corollary 16).

8. *Design containment with conditional bounds.* Parallel interfaces add their crossing ceilings; serial stages multiply only conditional ceilings (Theorems 18 and 20). A benchmark pass rate is not a crossing bound for routed faults.

9. *Let workers idle.* With optional participation, adding workers cannot lower the best achievable throughput (Proposition 21). A falling curve indicates a participation policy to change.

10. *Test substitutions per resource and per period.* A mechanization that lowers total demand can still overload one period or one resource (Proposition 23, Example 24).

11. *State queueing assumptions.* Queueing formulas need arrival and service assumptions that the error-law invariance does not supply. Do not use severity as a holding-cost rate without a reason (Proposition 25).

12. *Schedule memory, not memory-time.* Verification jobs with memory footprints need cumulative scheduling constraints; aggregate memory-time is a relaxation (Proposition 27, Example 28).

13. *Stabilize over admissible configurations, and budget separately.* MaxWeight over configurations built from admissible options keeps queues stable when demand lies strictly inside the capacity region. It does not enforce a lifetime risk budget (Theorem 30).

8 Related work

Allocating inspection effort is an old problem. Lindsay and Bishop place screening inspections on a serial production line by dynamic programming [53], and surveys cover the field [74, 61]. Testing effort has been allocated across software modules under a reliability-growth law [68]. These models assume a known fault law; our admissible sets use only bounds. Chance-constrained programming requires a constraint to hold with a stated probability under the decision taken [21]. Our admissibility is a worst-case premise, and it must hold under the chosen concurrency and review arrangement.

For AI systems, learning to defer routes cases from a model to a human [60, 63]. Some of this work adds an explicit limit on human workload [69, 9] or congestion at the human queue [59]. Lin and Zhang model agent-generated code changes that may pass an AI reviewer before a human maintainer approves them, as a queueing network with three resource pools [54]. They plan with a fluid linear program, use its dual prices, show that the review policy changes at staffing breakpoints and cannot be produced by one fixed ranking, count at most three split categories at an extreme point, as in Proposition 12, and prove a policy asymptotically optimal. Their model is the closest to Sections 3 and 6. It takes the false-acceptance and false-rejection rates as parameters. We first filter options by admissibility from bounds; when the remaining program data are law-independent or replaced by common uniform bounds, the allocation then does not depend on the error law (Theorem 2). We also add the review-set and containment results. AI control spends a limited human audit budget against a possibly adversarial model [36, 35], and language-model routers trade cost against quality for each query [22, 71]. Drift-plus-penalty methods extend MaxWeight with time-average constraints [64]; they are a possible route to the lifetime budget that Theorem 30 does not enforce.

Requiring feasibility under the decision actually chosen is itself standard: stochastic and robust optimization with decision-dependent uncertainty evaluate constraints against the uncertainty that the chosen decision induces [31, 65]. What we add is narrow: the application of that requirement to gated work, where the uncertain premise is admissibility under the chosen concurrency and review arrangement; the conditions under which the classical tools cited above then apply; and counterexamples where they fail.

9 Limitations

The proofs are by hand; no proof assistant checked them. The finite checks of Section 10 test instances and counterexamples; they do not replace the proofs. The fluid model is a model: executable schedules need the constraints of Proposition 27 and Theorem 30. Real workloads are not shown to satisfy any queueing or arrival assumption used here. Admissibility enters as an input per class under a stated reading; it must hold under the chosen concurrency and routing, and this paper does not measure it. The results do not estimate any capacity, price, coverage, or severity for a real system.

Two directions are not addressed here. The first is measurement design: how to allocate a limited audit budget to estimate the bounds that the scheduler relies on. Information lower bounds for adaptive sampling are a starting point [45]. The second is feedback: a policy decision changes constraints, and the allocation changes demand in turn. Contraction arguments give a unique equilibrium and a Lipschitz sensitivity when they apply [13], but LP-based allocations can jump at ties, so continuity must be established, not assumed. The principal as an executor is treated here only through Corollary 16 and Proposition 17; a model of intent elicitation is needed before more can be said. This is a preprint. It has not been peer reviewed. The prior-art search was targeted, not systematic.

10 Reproducibility

The script `simulate_allocation.py` reproduces every numerical claim and counterexample in this paper with exact rational arithmetic and the Python standard library. It prints PASS or FAIL for each claim and exits with a nonzero status on any failure. It also checks random instances: Hall's condition against maximum flow, the price certificate against direct feasibility, strong duality and the reduced-cost test on a small LP, the fractional and integer knapsack guarantees, submodularity and the greedy guarantee for fixed-law review value, the modularity bound by exact enumeration on a small graph, and the MaxWeight drift inequality. Four negative controls assert false claims (the shared gate, the integer ratio rule, the degenerate dual, and serial stages from marginals) and must be rejected.

Quantity	Value
Remark 5: principal-review ceiling, $f = 0.1$ / $f = 0.05$	240 / 480
Example 9: $\min_a p^\top D_a$ versus $p^\top \kappa$ at $p = (1, 1)$	2 versus 1.2
Example 11: reduced cost at $y = p = 1$; old and new optimum	-1; 0 and 0
Proposition 13(c): undiscarded greedy / optimum	1 / 3
Theorem 15: robust values of $\varnothing, \{1\}, \{2\}, \{1, 2\}$	0, 0, 0, 1
Proposition 21: n^* for $\alpha = 0.1$, $\beta = 0.01$	$\sqrt{90} \approx 9.49$
Example 24: total demand old / new; period-one demand	20 / 15; $15 > 10$
Example 28: aggregate memory-time / capacity	9 / 10

Code and data availability

The standard-library Python script `simulate_allocation.py` reproduces every numerical claim of Section 10. It is at https://github.com/ivmat/gated-computation-sim. This paper is archived on Zenodo, DOI 10.5281/zenodo.23449199.

References

[1] I. Matijašević. A Fault-Tolerance Threshold for Gated Agentic Computation. 2026. https://doi.org/10.5281/zenodo.20820968.

[2] I. Matijašević. Generated Gates Inherit Their Generator's Blind Spots. 2026. https://doi.org/10.5281/zenodo.20837102.

[3] I. Matijašević. Escape, Cost, and Correlation at the Verification Floor of Gated Agentic Computation. 2026. https://doi.org/10.5281/zenodo.21456783.

[4] I. Matijašević. Progressive Semantic Mechanization. 2026. https://doi.org/10.5281/zenodo.23119225.

[5] I. Matijašević. Beyond Constant Fault Rates: Reliability Bounds for Gated Computation and Semantic Mechanization. Preprint, 2026. https://doi.org/10.5281/zenodo.23293740.

[6] I. Matijašević. Risk-Gated Mechanization: Severity, Verifiability, and Decisions Invariant to the Executor's Error Law. Preprint, 2026. https://doi.org/10.5281/zenodo.23324667.

[7] A. Aggoun and N. Beldiceanu. Extending CHIP in order to solve complex scheduling and placement problems. *Mathematical and Computer Modelling* 17(7):57–73, 1993.

[8] M. Akgül. A note on shadow prices in linear programming. *Journal of the Operational Research Society* 35(5):425–431, 1984.

[9] J. V. Alves, D. Leitão, S. Jesus, M. O. P. Sampaio, J. Liébana, P. Saleiro, M. A. T. Figueiredo, and P. Bizarro. Cost-sensitive learning to defer to multiple experts with workload constraints. 2024. https://arxiv.org/abs/2403.06906.

[10] G. M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. *AFIPS Spring Joint Computer Conference*, pp. 483–485, 1967.

[11] D. C. Aucamp and D. I. Steinberg. The computation of shadow prices in linear programming. *Journal of the Operational Research Society* 33(6):557–565, 1982.

[12] N. T. J. Bailey. On queueing processes with bulk service. *Journal of the Royal Statistical Society, Series B* 16(1):80–87, 1954.

[13] S. Banach. Sur les opérations dans les ensembles abstraits et leur application aux équations intégrales. *Fundamenta Mathematicae* 3:133–181, 1922.

[14] P. Baptiste, C. Le Pape, and W. Nuijten. *Constraint-Based Scheduling*. Kluwer, 2001.

[15] A. Ben-Tal and A. Nemirovski. Robust solutions of uncertain linear programs. *Operations Research Letters* 25(1):1–13, 1999.

[16] D. Bertsimas and J. N. Tsitsiklis. *Introduction to Linear Optimization*. Athena Scientific, 1997.

[17] B. W. Boehm. Verifying and validating software requirements and design specifications. *IEEE Software* 1(1):75–88, 1984.

[18] S. Boyd and L. Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.

[19] C. Buyukkoc, P. Varaiya, and J. Walrand. The $c\mu$ rule revisited. *Advances in Applied Probability* 17(1):237–238, 1985.

[20] J. P. Buzen. Fundamental operational laws of computer system performance. *Acta Informatica* 7(2):167–182, 1976.

[21] A. Charnes and W. W. Cooper. Chance-constrained programming. *Management Science* 6(1):73–79, 1959.

[22] L. Chen, M. Zaharia, and J. Zou. FrugalGPT: how to use large language models while reducing cost and improving performance. 2023. https://arxiv.org/abs/2305.05176.

[23] D. R. Cox and W. L. Smith. *Queues*. Methuen, 1961.

[24] G. B. Dantzig. Discrete-variable extremum problems. *Operations Research* 5(2):266–288, 1957.

[25] P. J. Denning and J. P. Buzen. The operational analysis of queueing network models. *ACM Computing Surveys* 10(3):225–261, 1978.

[26] D. E. Eckhardt and L. D. Lee. A theoretical basis for the analysis of multiversion software subject to coincident errors. *IEEE Transactions on Software Engineering* SE-11(12):1511–1517, 1985.

[27] J. D. Esary and F. Proschan. Coherent structures of non-identical components. *Technometrics* 5(2):191–209, 1963.

[28] J. Farkas. Theorie der einfachen Ungleichungen. *Journal für die reine und angewandte Mathematik* 124:1–27, 1902.

[29] U. Feige. A threshold of $\ln n$ for approximating set cover. *Journal of the ACM* 45(4):634–652, 1998.

[30] L. R. Ford and D. R. Fulkerson. Maximal flow through a network. *Canadian Journal of Mathematics* 8:399–404, 1956.

[31] V. Goel and I. E. Grossmann. A class of stochastic programs with decision dependent uncertainty. *Mathematical Programming* 108:355–394, 2006.

[32] T. Gal. Shadow prices and sensitivity analysis in linear programming under degeneracy. *OR Spektrum* 8(2):59–71, 1986.

[33] D. Gale. A theorem on flows in networks. *Pacific Journal of Mathematics* 7(2):1073–1082, 1957.

[34] L. Georgiadis, M. J. Neely, and L. Tassiulas. Resource allocation and cross-layer control in wireless networks. *Foundations and Trends in Networking* 1(1):1–144, 2006.

[35] C. Griffin, L. Thomson, B. Shlegeris, and A. Abate. Games for AI control: models of safety evaluations of AI deployment protocols. 2024. https://arxiv.org/abs/2409.07985.

[36] R. Greenblatt, B. Shlegeris, K. Sachan, and F. Roger. AI control: improving safety despite intentional subversion. 2023. https://arxiv.org/abs/2312.06942.

[37] N. J. Gunther. *Guerrilla Capacity Planning*. Springer, 2007.

[38] N. J. Gunther. A general theory of computational scalability based on rational functions. 2008. https://arxiv.org/abs/0808.1431.

[39] P. Hall. On representatives of subsets. *Journal of the London Mathematical Society* 10:26–30, 1935.

[40] M. Harchol-Balter. *Performance Modeling and Design of Computer Systems: Queueing Theory in Action*. Cambridge University Press, 2013.

[41] M. Hiller, A. Jhumka, and N. Suri. An approach for analysing the propagation of data errors in software. *Proceedings of the International Conference on Dependable Systems and Networks*, pp. 161–170, 2001.

[42] M. Hiller, A. Jhumka, and N. Suri. EPIC: profiling the propagation and effect of data errors in software. *IEEE Transactions on Computers* 53(5):512–530, 2004.

[43] M. Iri. On an extension of the maximum-flow minimum-cut theorem to multicommodity flows. *Journal of the Operations Research Society of Japan* 13(3):129–135, 1971.

[44] B. Jansen, J. J. de Jong, C. Roos, and T. Terlaky. Sensitivity analysis in linear programming: just be careful! *European Journal of Operational Research* 101(1):15–28, 1997.

[45] E. Kaufmann, O. Cappé, and A. Garivier. On the complexity of best-arm identification in multi-armed bandit models. *Journal of Machine Learning Research* 17:1–42, 2016.

[46] H. Kellerer, U. Pferschy, and D. Pisinger. *Knapsack Problems*. Springer, 2004.

[47] J. F. C. Kingman. The single server queue in heavy traffic. *Mathematical Proceedings of the Cambridge Philosophical Society* 57(4):902–904, 1961.

[48] J. F. C. Kingman. On queues in heavy traffic. *Journal of the Royal Statistical Society, Series B* 24(2):383–392, 1962.

[49] T. C. Koopmans. Identification problems in economic model construction. *Econometrica* 17(2):125–144, 1949.

[50] A. Krause, H. B. McMahan, C. Guestrin, and A. Gupta. Robust submodular observation selection. *Journal of Machine Learning Research* 9:2761–2801, 2008.

[51] J. H. Lala and R. E. Harper. Architectural principles for safety-critical real-time applications. *Proceedings of the IEEE* 82(1):25–40, 1994.

[52] L. Le Cam. Convergence of estimates under dimensionality restrictions. *Annals of Statistics* 1(1):38–53, 1973.

[53] G. F. Lindsay and A. B. Bishop. Allocation of screening inspection effort—a dynamic-programming approach. *Management Science* 10(2):342–352, 1964.

[54] R. Lin and J. Zhang. When should AI review precede human approval of agent-generated code? 2026, version 2; version 1 was titled "When to screen, when to bypass: LLM-judges in resource-scarce AI-human workflow". https://arxiv.org/abs/2603.13870.

[55] B. Littlewood and D. R. Miller. Conceptual modeling of coincident failures in multiversion software. *IEEE Transactions on Software Engineering* 15(12):1596–1614, 1989.

[56] Center for Chemical Process Safety. *Layer of Protection Analysis: Simplified Process Risk Assessment*. American Institute of Chemical Engineers, 2001.

[57] J. K. Lenstra, D. B. Shmoys, and É. Tardos. Approximation algorithms for scheduling unrelated parallel machines. *Mathematical Programming* 46:259–271, 1990.

[58] H. Luss. Operations research and capacity expansion problems: a survey. *Operations Research* 30(5):907–947, 1982.

[59] T. Lykouris and W. Weng. Learning to defer in congested systems: the AI-human interplay. 2024. https://arxiv.org/abs/2402.12237.

[60] D. Madras, T. Pitassi, and R. Zemel. Predict responsibly: improving fairness and accuracy by learning to defer. 2017. https://arxiv.org/abs/1711.06664.

[61] S. S. Mandroli, A. K. Shrivastava, and Y. Ding. A survey of inspection strategy and sensor distribution studies in discrete-part manufacturing processes. *IIE Transactions* 38(4):309–328, 2006.

[62] H. D. Mills. Marginal values of matrix games and linear programs. In H. W. Kuhn and A. W. Tucker (eds.), *Linear Inequalities and Related Systems*, Annals of Mathematics Studies 38, pp. 183–194. Princeton University Press, 1956.

[63] H. Mozannar and D. Sontag. Consistent estimators for learning to defer to an expert. 2020. https://arxiv.org/abs/2006.01862.

[64] M. J. Neely. *Stochastic Network Optimization with Application to Communication and Queueing Systems*. Morgan & Claypool, 2010.

[65] O. Nohadani and K. Sharma. Optimization under decision-dependent uncertainty. *SIAM Journal on Optimization* 28(2):1773–1795, 2018.

[66] G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher. An analysis of approximations for maximizing submodular set functions—I. *Mathematical Programming* 14(1):265–294, 1978.

[67] M. F. Neuts. A general class of bulk queues with Poisson input. *Annals of Mathematical Statistics* 38(3):759–770, 1967.

[68] H. Ohtera and S. Yamada. Optimal allocation and control problems for software-testing resources. *IEEE Transactions on Reliability* 39(2):171–176, 1990.

[69] N. Okati, A. De, and M. Gomez-Rodriguez. Differentiable learning under triage. 2021. https://arxiv.org/abs/2103.08902.

[70] K. Onaga and O. Kakusho. On feasibility conditions of multicommodity flows in networks. *IEEE Transactions on Circuit Theory* 18(4):425–429, 1971.

[71] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica. RouteLLM: learning to route LLMs with preference data. 2024. https://arxiv.org/abs/2406.18665.

[72] J. B. Orlin, A. S. Schulz, and R. Udwani. Robust monotone submodular function maximization. *Mathematical Programming* 172(1–2):505–537, 2018.

[73] A. A. B. Pritsker, L. J. Watters, and P. M. Wolfe. Multiproject scheduling with limited resources: a zero-one programming approach. *Management Science* 16(1):93–108, 1969.

[74] T. Raz. A survey of models for allocating inspection effort in multistage production systems. *Journal of Quality Technology* 18(4):239–247, 1986.

[75] A. L. Soyster. Convex programming with set-inclusive constraints and applications to inexact linear programming. *Operations Research* 21(5):1154–1157, 1973.

[76] L. Tassiulas and A. Ephremides. Stability properties of constrained queueing systems and scheduling policies for maximum throughput in multihop radio networks. *IEEE Transactions on Automatic Control* 37(12):1936–1948, 1992.

[77] A. C. Williams. Marginal values in linear programming. *Journal of the Society for Industrial and Applied Mathematics* 11(1):82–94, 1963.